

Grading

The final score that you will receive for your programming assignment is generated in relation to the total points set in your programming assignment item—not the total point value in the nbgrader notebook.

When calculating the final score shown to learners, the programming assignment takes the percentage of earned points vs. the total points provided by nbgrader and returns a score matching the equivalent percentage of the point value for the programming assignment.

DO NOT CHANGE VARIABLE OR METHOD SIGNATURES The autograder will not work properly if you change the variable or method signatures.

Support Vector Machines

In this assignment we'll explore the details of the Soft-Margin SVM and look at how the choice of tuning parameters affects the learned models. We'll also look at kernel SVMs for non-linearly separable and methods for choosing and visualizing good hyperparameters.

Note: Below are some helper functions which are used throughout this notebook. Execute these before continuing. Do not modify.

```
In [1]: import numpy as np
from sklearn.datasets import make_blobs
from matplotlib.colors import Normalize
import matplotlib.pyplot as plt
%matplotlib inline

def linear_plot(X, y, w=None, b=None):

    mycolors = {"blue": "steelblue", "red": "#a76c6e", "green": "#6a9373"}
    colors = [mycolors["red"] if yi==1 else mycolors["blue"] for yi in y]

    # Plot data
    fig, ax = plt.subplots(nrows=1, ncols=1, figsize=(8,8))
    ax.scatter(X[:,0], X[:,1], color=colors, s=150, alpha=0.95, zorder=2)

    # Plot boundaries
    lower_left = np.min([np.min(X[:,0]), np.min(X[:,1])])
    upper_right = np.max([np.max(X[:,0]), np.max(X[:,1])])
    gap = .1*(upper_right-lower_left)
    xplot = np.linspace(lower_left-gap, upper_right+gap, 20)
    if w is not None and b is not None:
        ax.plot(xplot, (-b - w[0]*xplot)/w[1], color="gray", lw=2, zorder=1)
        ax.plot(xplot, ( 1 -b - w[0]*xplot)/w[1], color="gray", lw=2, ls="--")
        ax.plot(xplot, (-1 -b - w[0]*xplot)/w[1], color="gray", lw=2, ls="--")
```

```

ax.set_xlim([lower_left-gap, upper_right+gap])
ax.set_ylim([lower_left-gap, upper_right+gap])

ax.grid(alpha=0.25)

def part2data():

    np.random.seed(1239)

    X = np.zeros((22,2))
    X[0:10,0] = 1.5*np.random.rand(10)
    X[0:10,1] = 1.5*np.random.rand(10)
    X[10:20,0] = 1.5*np.random.rand(10) + 1.75
    X[10:20,1] = 1.5*np.random.rand(10) + 1
    X[20,0] = 1.5
    X[20,1] = 2.25
    X[21,0] = 1.6
    X[21,1] = 0.25

    y = np.ones(22)
    y[10:20] = -1
    y[20] = 1
    y[21] = -1

    return X, y

def part3data(N=100, seed=1235):

    np.random.seed(seed)

    X = np.random.uniform(-1,1,(N,2))
    y = np.array([1 if y-x > 0 else -1 for (x,y) in zip(X[:,0]**2 * np.sin(2
    X = X + np.random.normal(0,.1,(N,2))

    return X, y

def nonlinear_plot(X, y, clf=None):

    mycolors = {"blue": "steelblue", "red": "#a76c6e", "green": "#6a9373"}

    fig, ax = plt.subplots(nrows=1, ncols=1, figsize=(10,10))

    colors = [mycolors["red"] if yi==1 else mycolors["blue"] for yi in y]
    ax.scatter(X[:,0],X[:,1], marker='o', color=colors, s=100, alpha=0.5)

    ax.arrow(-1.25,0,2.5,0, head_length=0.05, head_width=0.05, fc="gray", ec
    ax.arrow(0,-1.25,0,2.5, head_length=0.05, head_width=0.05, fc="gray", ec
    z = np.linspace(0.25,3.5,10)

    ax.set_xlim([-1.50,1.50])
    ax.set_ylim([-1.50,1.50])

    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

```

```

ax.spines['bottom'].set_visible(False)
ax.spines['left'].set_visible(False)
plt.xticks([], fontsize=16)
plt.yticks([], fontsize=16)

if clf:

    clf.fit(X,y)

    x_min = X[:, 0].min()+.00
    x_max = X[:, 0].max()-.00
    y_min = X[:, 1].min()+.00
    y_max = X[:, 1].max()-.00

    colors = [mycolors["red"] if yi==1 else mycolors["blue"] for yi in y

    XX, YY = np.mgrid[x_min:x_max:200j, y_min:y_max:200j]
    Z = clf.decision_function(np.c_[XX.ravel(), YY.ravel()])

    # Put the result into a color plot
    Z = Z.reshape(XX.shape)
    plt.contour(XX, YY, Z, colors=[mycolors["blue"], "gray", mycolors["red"], "gray"],
                levels=[-1.0, 0, 1.0], linewidths=[2,2,2], alpha=0.9)

class MidpointNormalize(Normalize):

    def __init__(self, vmin=None, vmax=None, midpoint=None, clip=False):
        self.midpoint = midpoint
        Normalize.__init__(self, vmin, vmax, clip)

    def __call__(self, value, clip=None):
        x, y = [self.vmin, self.midpoint, self.vmax], [0, 0.5, 1]
        return np.ma.masked_array(np.interp(value, x, y))

def plotSearchGrid(grid):

    scores = [x for x in grid.cv_results_["mean_test_score"]]
    scores = np.array(scores).reshape(len(grid.param_grid["C"]), len(grid.param_grid["gamma"]))

    plt.figure(figsize=(10, 8))
    plt.subplots_adjust(left=.2, right=0.95, bottom=0.15, top=0.95)
    plt.imshow(scores, interpolation='nearest', cmap=plt.cm.hot,
               norm=MidpointNormalize(vmin=0.2, midpoint=0.92))
    plt.xlabel('gamma')
    plt.ylabel('C')
    plt.colorbar()
    plt.xticks(np.arange(len(grid.param_grid["gamma"])), grid.param_grid["gamma"])
    plt.yticks(np.arange(len(grid.param_grid["C"])), grid.param_grid["C"])
    plt.title('Validation accuracy')
    plt.show()

from IPython.core.display import HTML
HTML("""
<style>

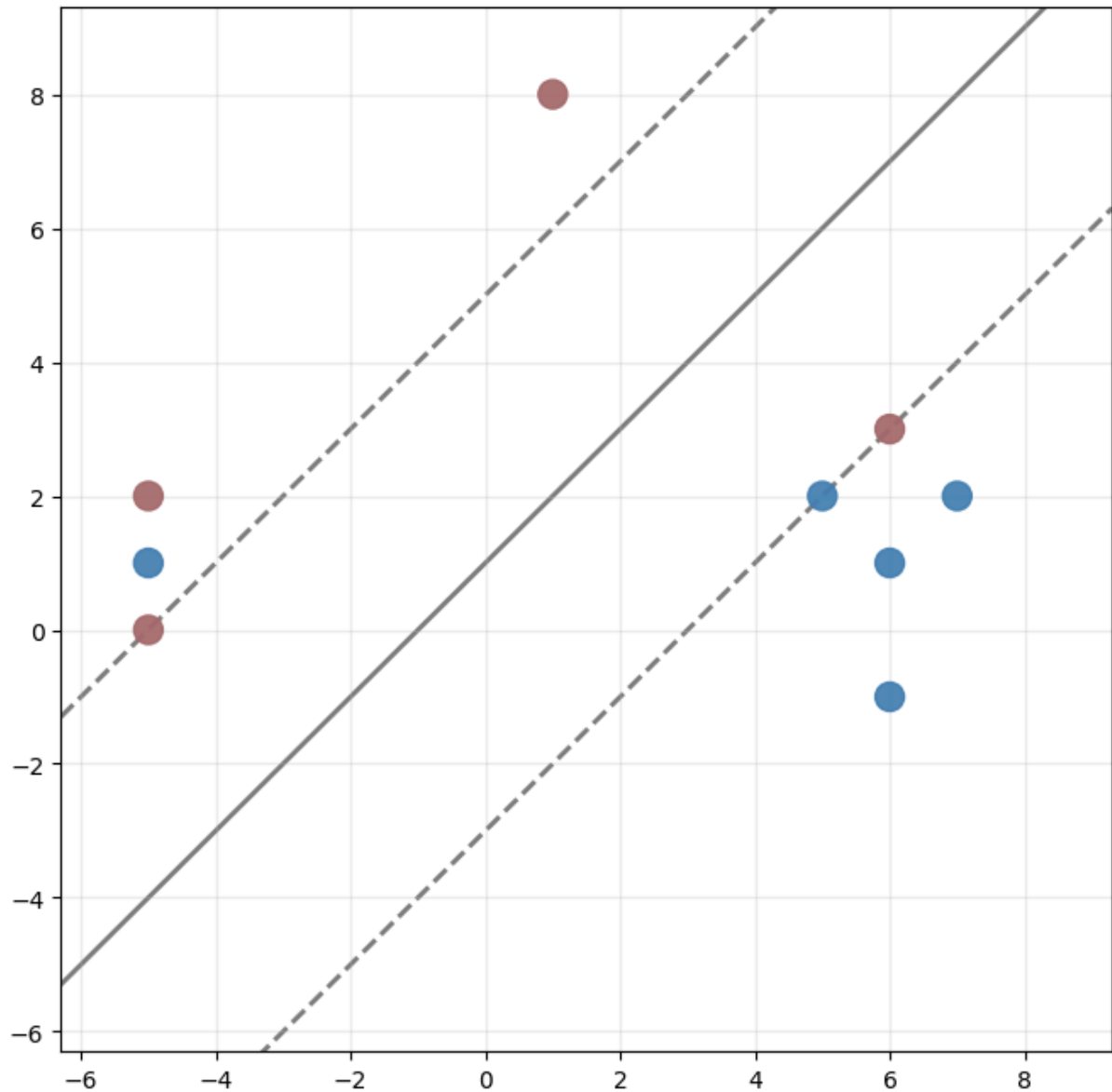
```

```
.MathJax nobr>span.math>span{border-left-width:0 !important};  
</style>  
""")
```

Out[1]:

Part 1: SVM [20 pts]

```
In [2]: import numpy as np  
  
# Data and Labels  
X = np.array([[1,8],[7,2],[6,-1],[-5,0], [-5,1], [-5,2],[6,3],[6,1],[5,2]])  
y = np.array([1,-1,-1,1,-1,1,1,-1,-1])  
  
# Support vector parameters  
w, b = np.array([-1/4, 1/4]), -1/4  
  
# Plot the data and support vector boundaries  
linear_plot(X, y, w=w, b=b)
```



Part A [5 pts]: What is the margin of this particular SVM?

```
In [3]: margin = 1 / np.linalg.norm(w) # update margin to be a correct number
        # YOUR CODE HERE
        #raise NotImplementedError()
```

```
In [4]: # tests that margin was updated correctly
```

Part B [5 pts]: Which training examples are the support vectors? Assign the coordinate in the list. e.g. support_vectors = [(1,0),(0,0)]

```
In [5]: # uncomment and update support_vectors
        # YOUR CODE HERE
        # raise NotImplementedError()
        support_vectors=[(-5,1),(-5,2),(6,3),(6,1),(5,2)]
```

```
In [ ]: # tests support_vectors
```

Explain your answer for which training examples are support vectors. For the support vectors that you have identified, which are the in-bound support vectors and which are the out-bound support vectors?

Support vectors satisfy either $X @ w + b = 1$ or -1

In-bound support vectors: $(-5,2), (6,3), (6,1), (5,2)$

Out-bound support vectors: $(-5,1)$

```
In [ ]: # instructor testing cell
# YOUR CODE HERE
raise NotImplementedError()
```

Part C [5 pts]: Which training examples have nonzero slack? List their coordinates.

```
In [7]: # list of coordinates with nonzero slack
nonzero_slack=[(-5,0),(-5,2),(6,3),(6,1),(5,2)]
# YOUR CODE HERE
#raise NotImplementedError()
```

```
In [ ]: # tests nonzero_slack list
```

How did you know which training examples had nonzero slack?

The training examples with nonzero slack are either misclassified [none of them] or are inside the margin $[(6,-1),(-5,1),(-5,2),(6,1),(5,2)]$

```
In [ ]: # instructor testing cell
# YOUR CODE HERE
raise NotImplementedError()
```

Part D [5 pts]: Compute the slack ξ_i associated with the misclassified points.

```
In [8]: # YOUR CODE HERE
#raise NotImplementedError()
# compute the slack associated with the misclassified points
# return a list slacks with the slack computations

# there aren't any misclassified points

slack1 = []
slack2 = []

slacks = [slack1, slack2]

print(slacks)
```

```
[[ ], [ ]]
```

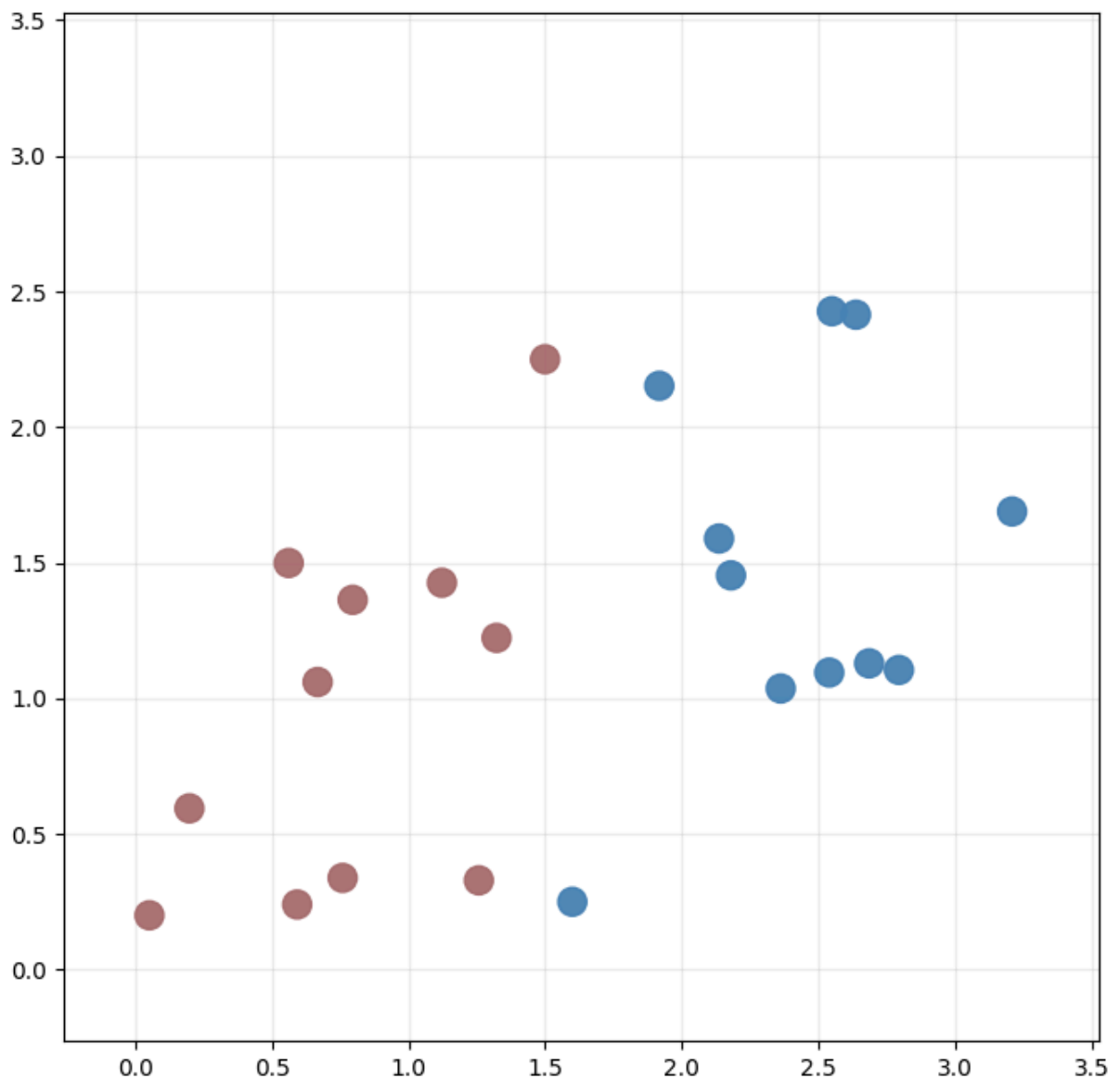
```
In [ ]: # tests slacks
```

Do these slack values agree with the plot of the data and the support vector boundaries?

Part 2: The Margin vs Slack [20 pts]

In this problem we'll figure out how to fit linear SVM models to data using sklearn. Consider the data shown below.

```
In [19]: X, y = part2data()  
         linear_plot(X, y)
```



Part A [5 pts]: Let's fit a linear Soft-Margin SVM to the data above. For SVMs with a linear kernel we'll use the `LinearSVM` method from sklearn's `svm` module. Go now

and look at the documentation.

Recall that the primal objective function for the linear kernel SVM is as follows

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i^p.$$

Note that the C parameter definition in sklearn is different from the textbook.

The two optional parameters in `LinearSVM` that we'll be most concerned with are `C`, the hyperparameter weighting the slackness contribution to the primal objective function, and `loss`, which determines the exponent on the slack variables in the sum. When $p = 1$ from above equation, the loss is hinge loss, whereas is $p = 2$, it's $L2$ form, the loss is called squared-hinge.

Write some code below to train a linear SVM with $C = 1$ and $p = 1$, get the computed weight vector and bias, and the plot the resulting model.

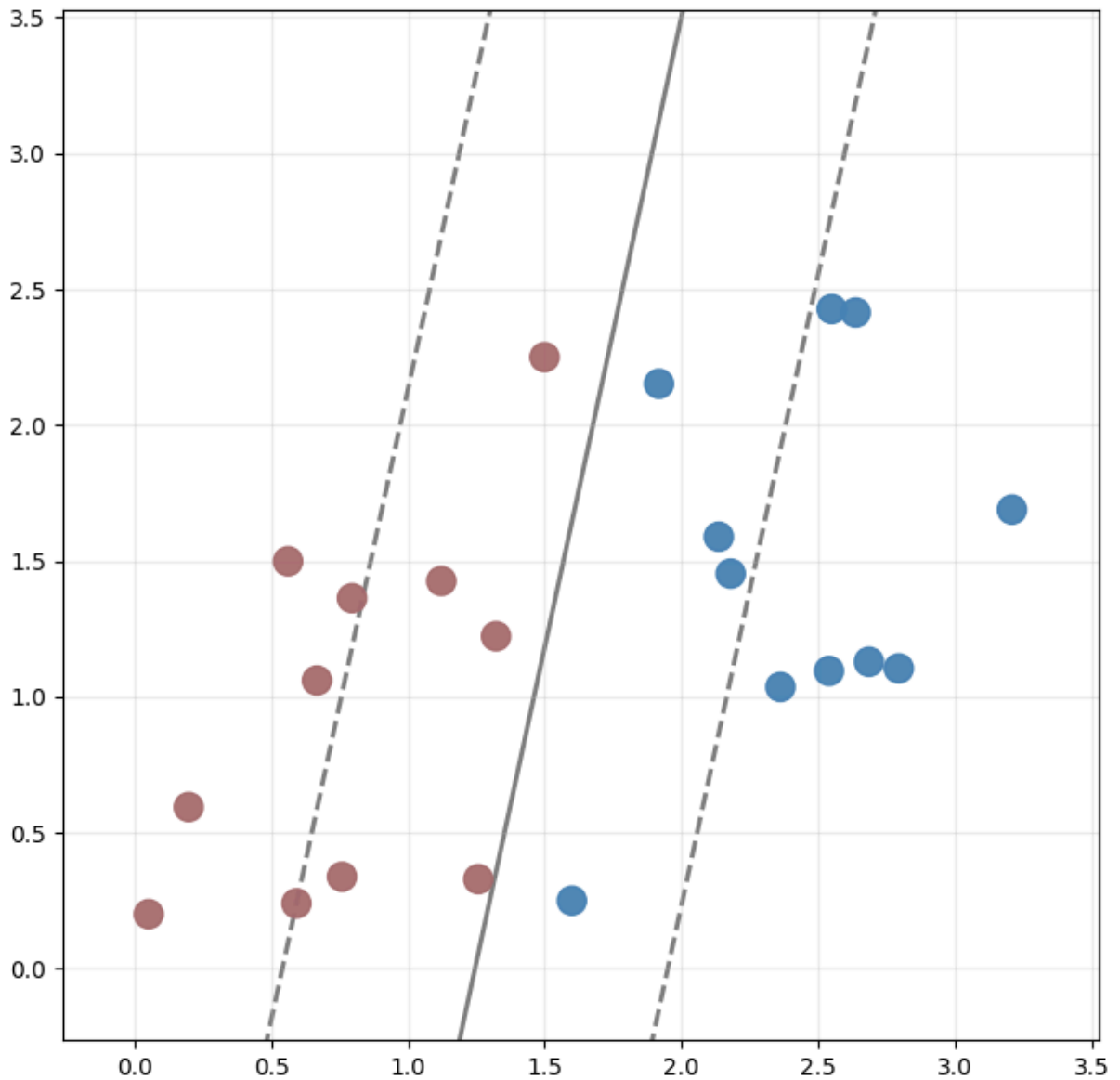
```
In [21]: from sklearn.svm import LinearSVC

# TODO: Build a LinearSVC model called lsvm. Train the model and get the par
X, y = part2data()

lsvm = LinearSVC(C=1, loss='hinge', max_iter=10000)
# YOUR CODE HERE
#raise NotImplementedError()
lsvm.fit(X, y)

# use this code to plot the resulting model
w = lsvm.coef_[0]
b = lsvm.intercept_
print(w,b)
linear_plot(X, y, w=w, b=b)
```

```
[-1.41897528  0.30584252] [1.76759606]
```

```
In [ ]: # tests w and b
```

Part B [5 pts]: Experiment with different values of `C`. How does the choice of `C` affect the nature of the decision boundary and the associated margin?

A smaller `C` value (0.01) has a wider margin and more tolerance for misclassification. A larger `C` value (100) has a narrower margin, less slack, and more potential for overfitting.

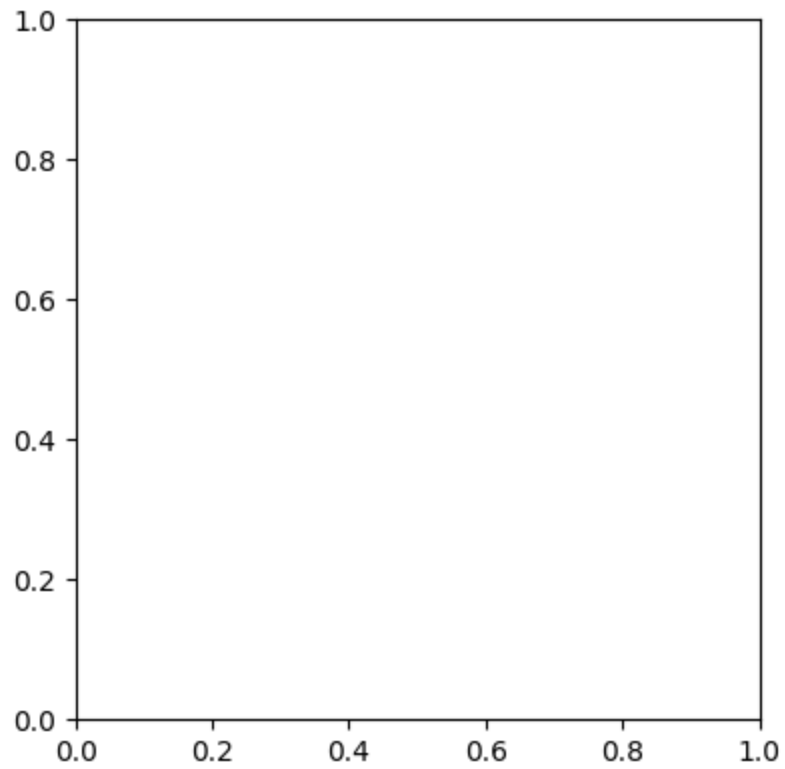
```
In [22]: from sklearn.svm import LinearSVC

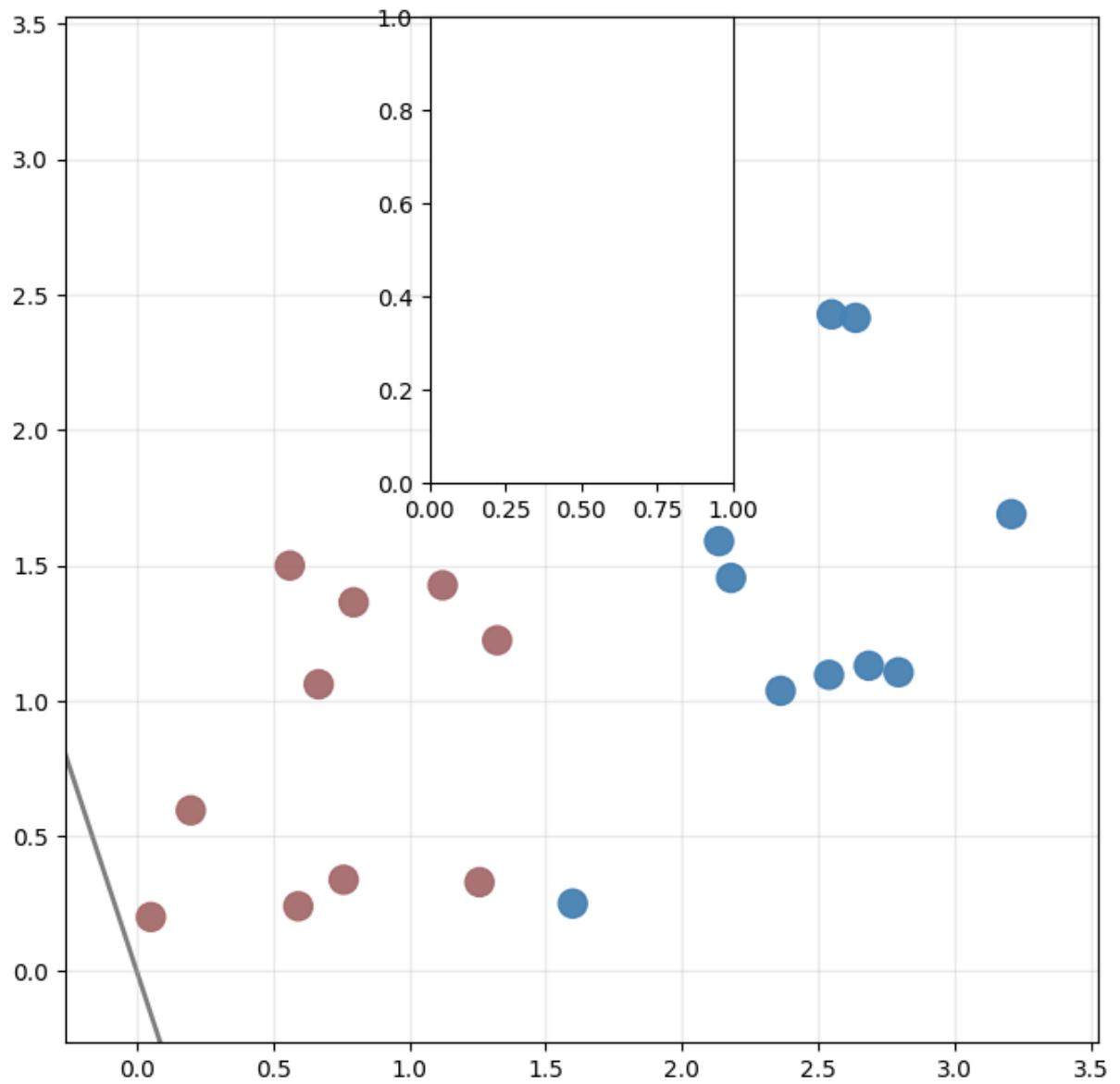
# TODO: Change the svm model parameter and plot the result.

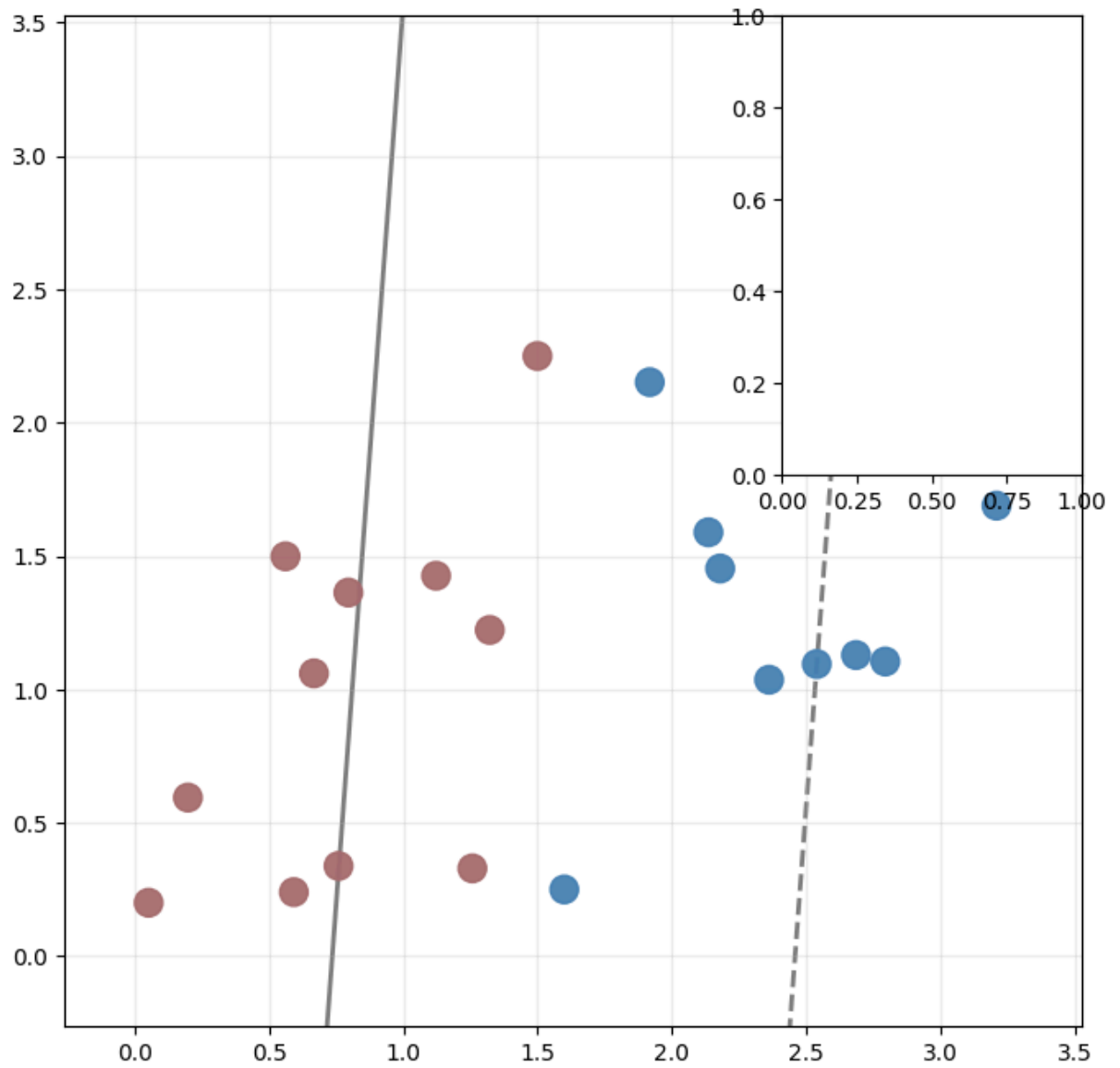
# YOUR CODE HERE
#raise NotImplementedError()

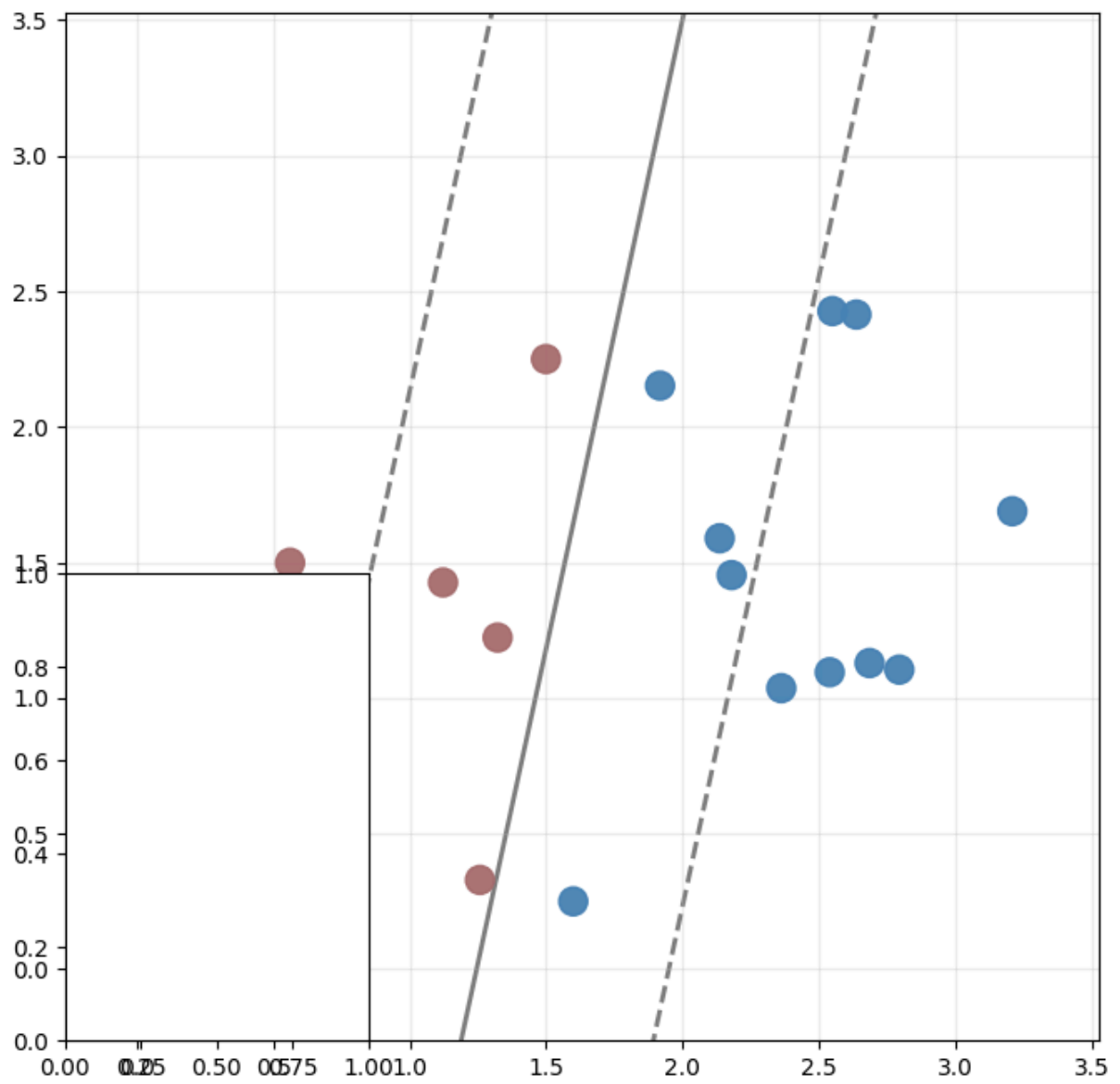
C_values = [0.01, 0.1, 1, 10, 100]
plt.figure(figsize=(15,10))
```

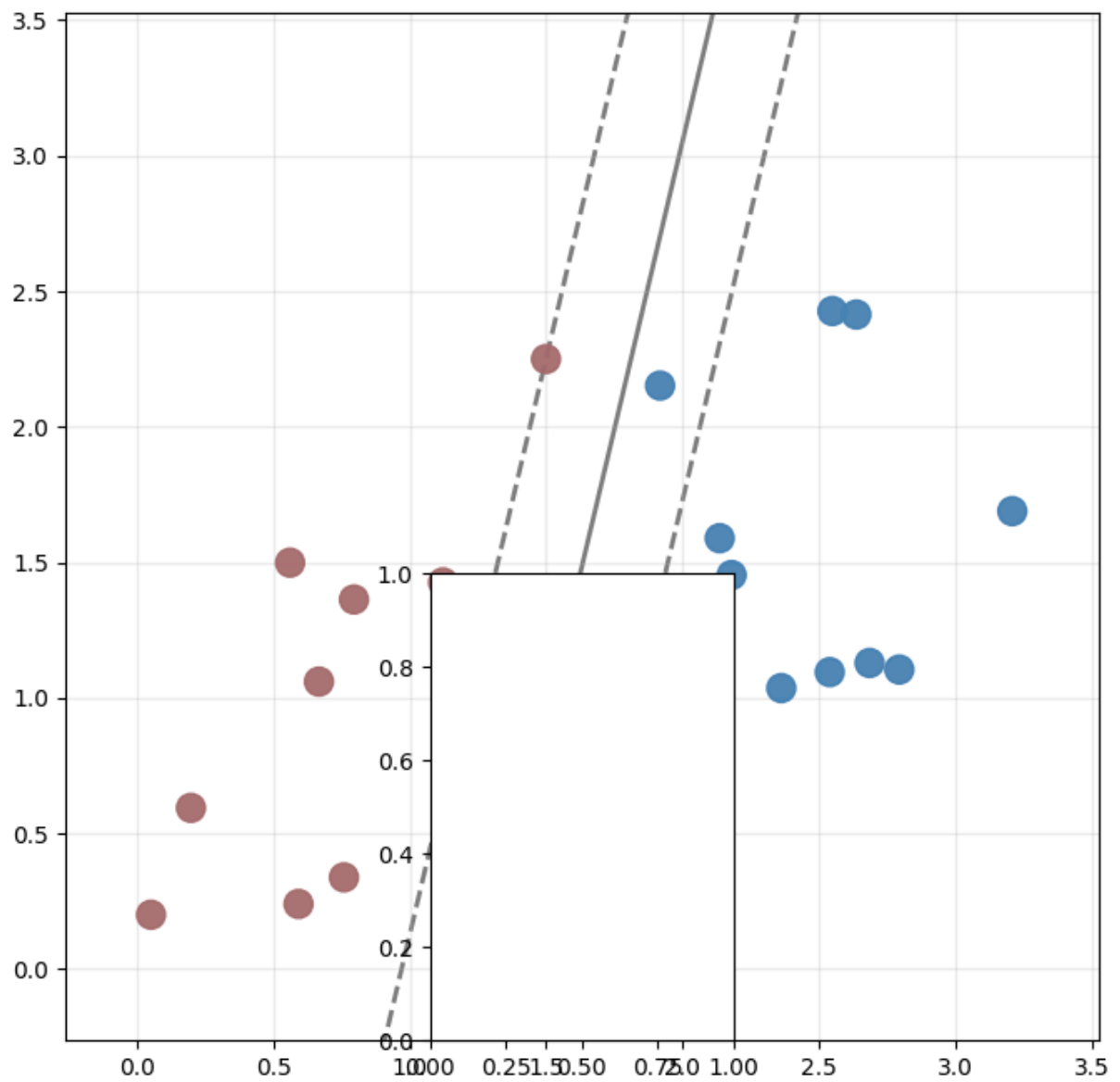
```
for i, C in enumerate(C_values):  
    lsvm = LinearSVC(C=C, loss='hinge', max_iter=10000)  
    lsvm.fit(X, y)  
    w = lsvm.coef_[0]  
    b = lsvm.intercept_[0]  
  
    plt.subplot(2, 3, i+1)  
    linear_plot(X, y, w=w, b=b)
```

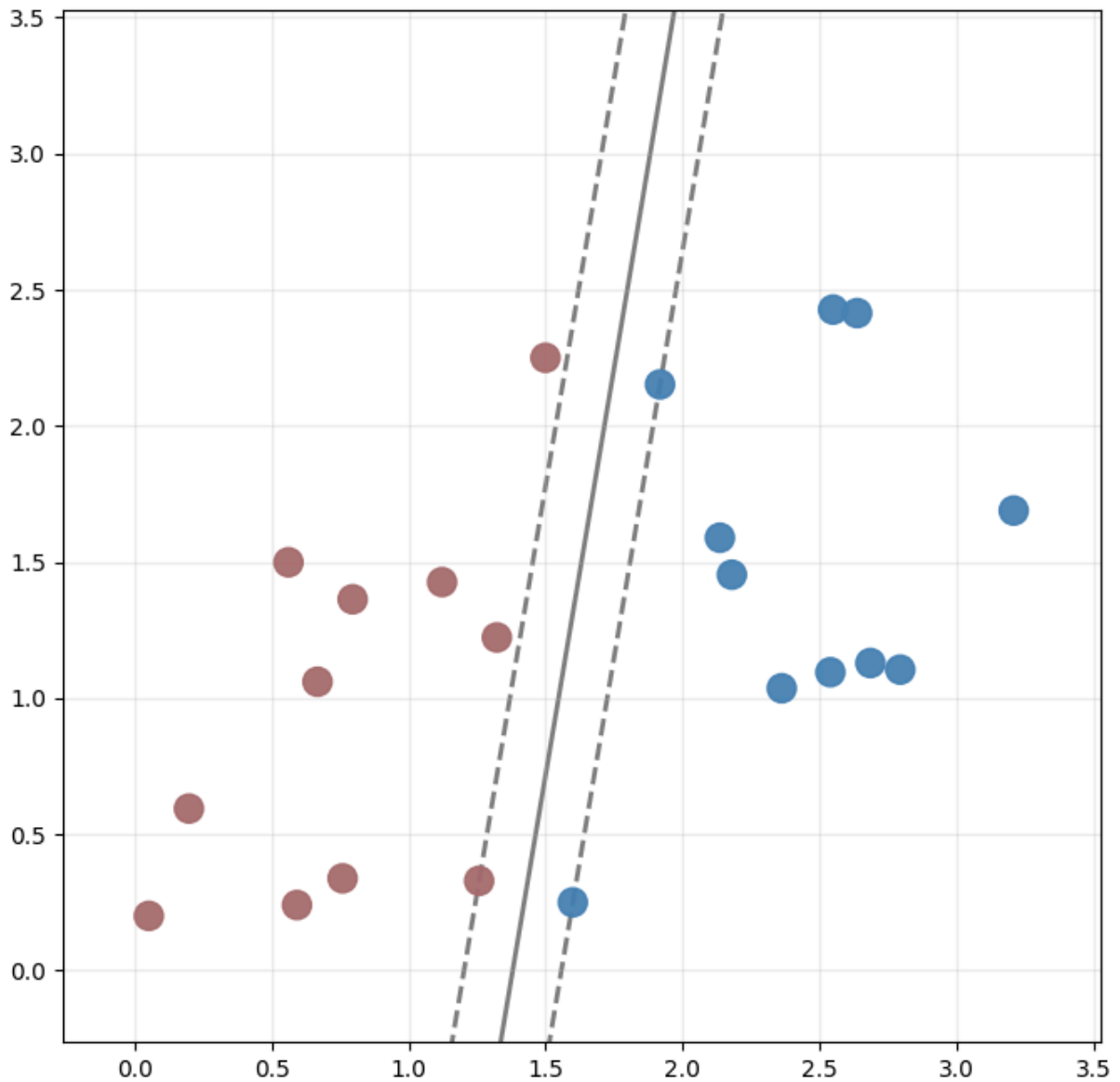












Part C [5 pts]: Set `C=3` and compare the results you get when using the `hinge` vs the `squared_hinge` values for the `loss` parameter. Explain your observations. Compare hinge loss vs. squared hinge loss.

Hinge loss penalizes only when the margin is violated. Squared hinge loss penalizes violations quadratically, so it's more sensitive to outliers. The decision boundaries may differ subtly; squared hinge tends to produce larger weights, so there's more penalty for larger slacks.

```
In [23]: from sklearn.svm import LinearSVC

# TODO: Train the model and get the parameters, pay attention to the loss parameter
# YOUR CODE HERE
#raise NotImplementedError()

lsvm_hinge = LinearSVC(C=3, loss='hinge', max_iter=10000)
lsvm_hinge.fit(X, y)
w1 = lsvm_hinge.coef_[0]
```

```
b1 = lsvm_hinge.intercept_[0]

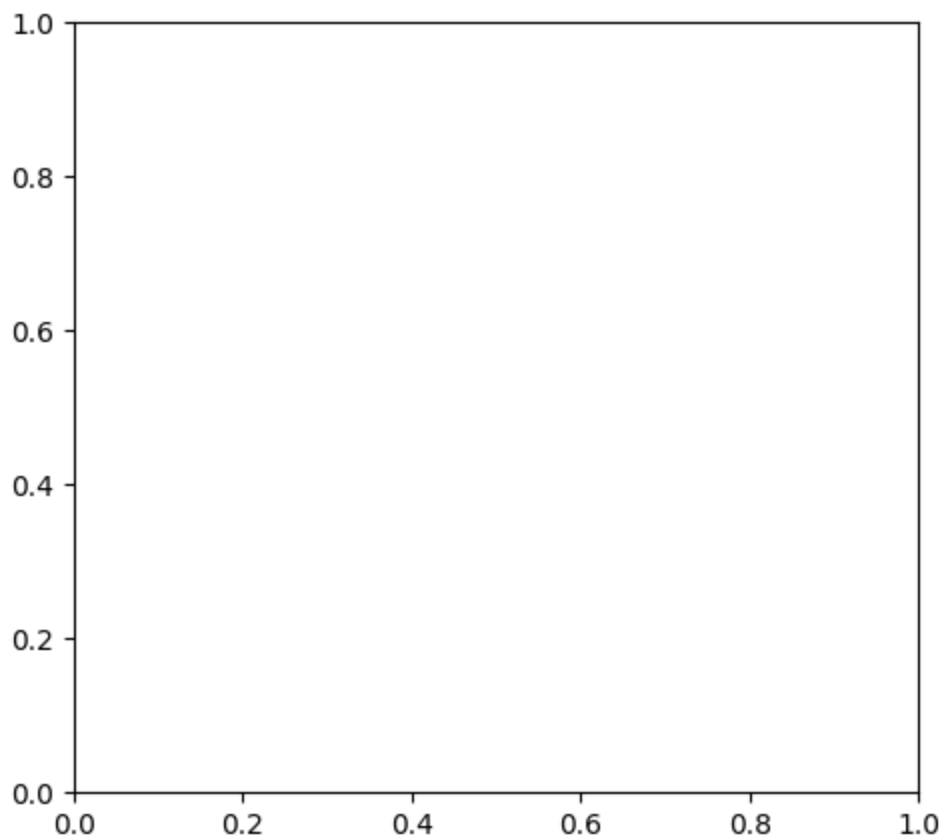
lsvm_sqhinge = LinearSVC(C=3, loss='squared_hinge', max_iter=10000)
lsvm_sqhinge.fit(X, y)
w2 = lsvm_sqhinge.coef_[0]
b2 = lsvm_sqhinge.intercept_[0]

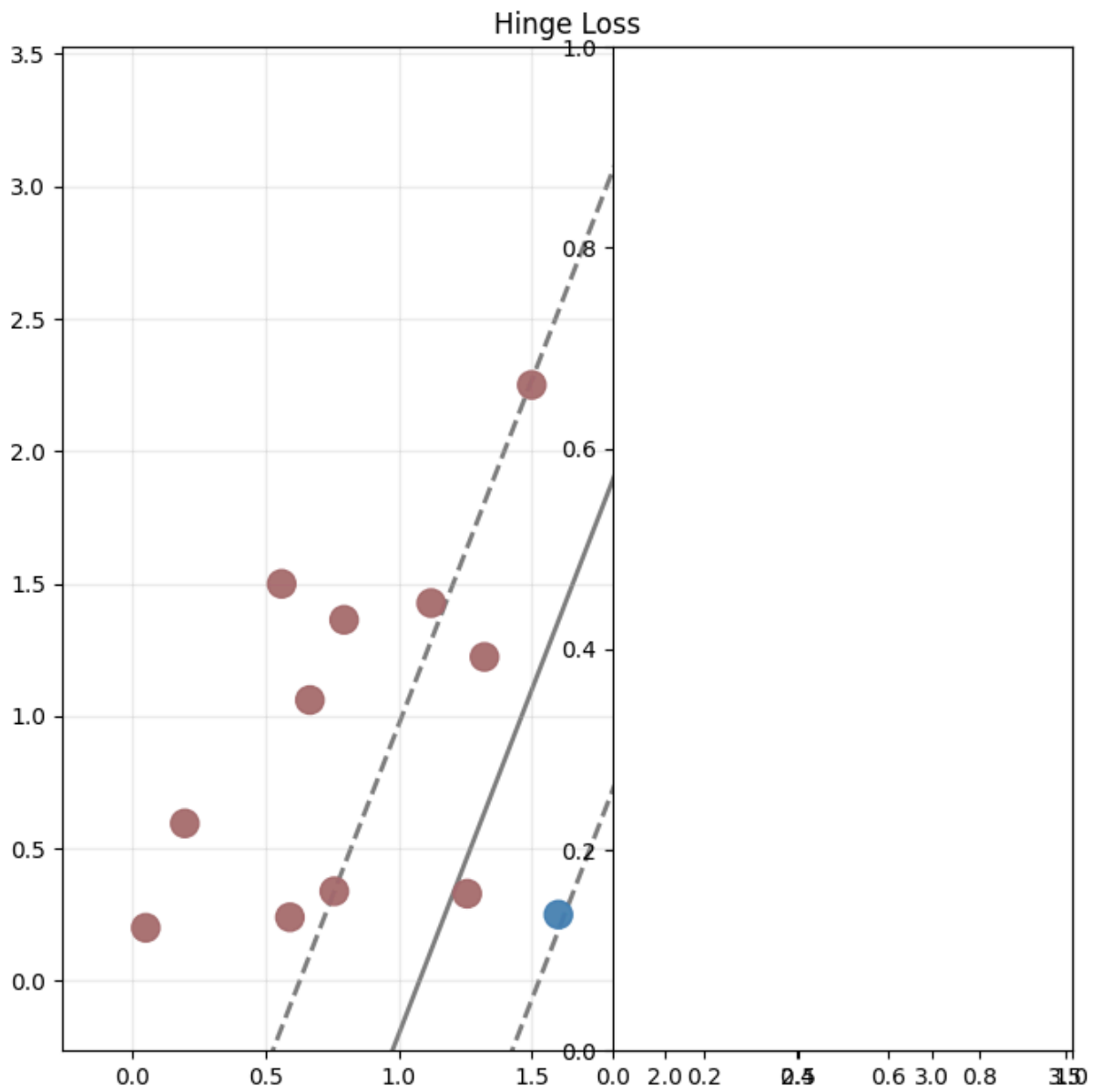
plt.figure(figsize=(12,5))

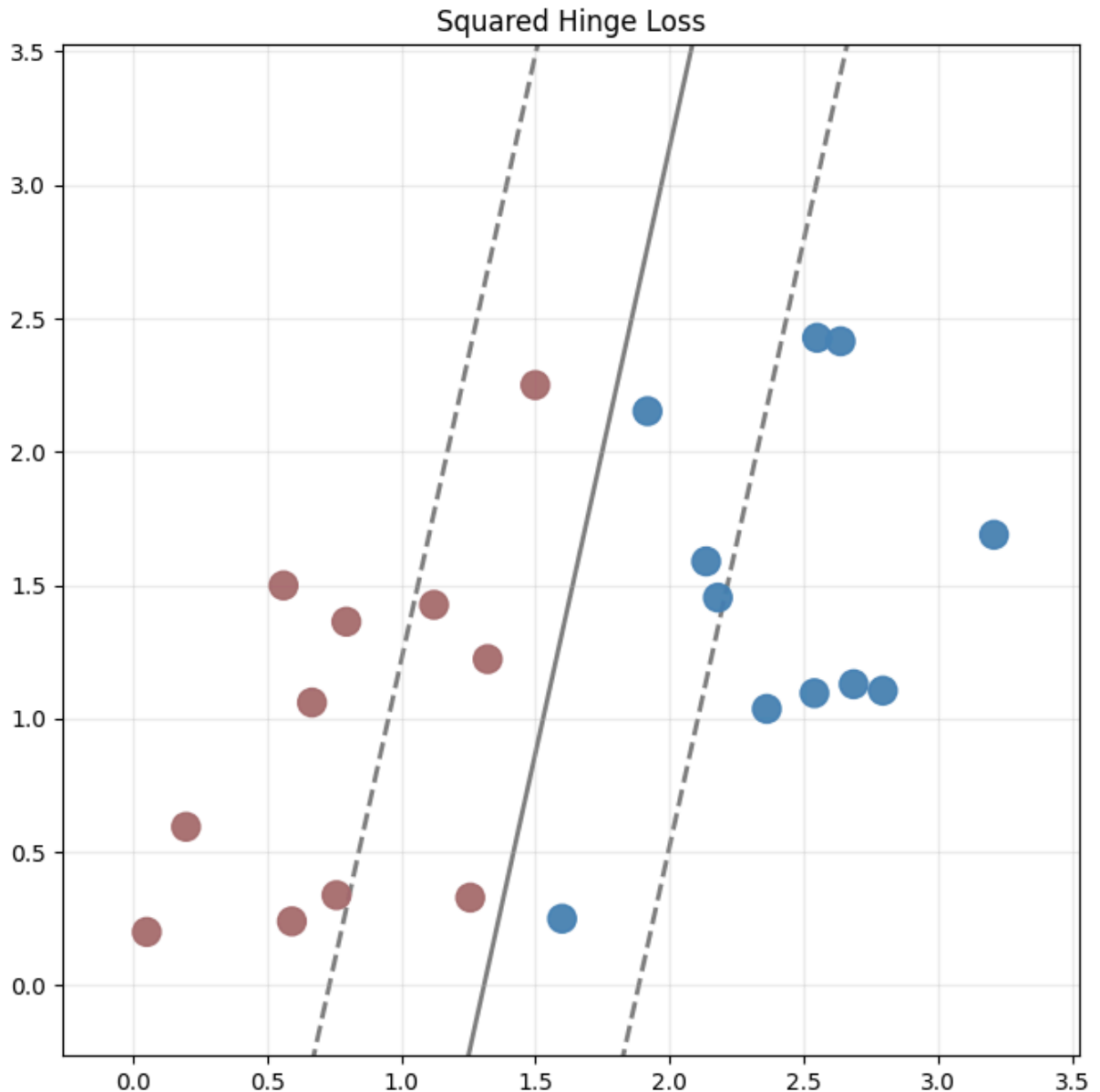
plt.subplot(1, 2, 1)
linear_plot(X, y, w=w1, b=b1)
plt.title("Hinge Loss")

plt.subplot(1, 2, 2)
linear_plot(X, y, w=w2, b=b2)
plt.title("Squared Hinge Loss")
```

Out[23]: Text(0.5, 1.0, 'Squared Hinge Loss')







Part D [5 pts]: In general, how does the choice of `C` affect the bias and variance of the model?

A smaller `C` value places more emphasis on maximizing the margins and allows more slack, so there's higher bias and lower variance. A larger `C` penalizes misclassification more, so it leads to smaller margin and stricter decision boundary, so there's lower bias and higher variance.

```
In [ ]: # YOUR CODE HERE
raise NotImplementedError()
```

Part 3: Nonlinear SVM, Parameter Tuning, Accuracy, and Cross-Validation [25 pts]

Any support vector machine classifier will have at least one parameter that needs to be tuned based on the training data. The guaranteed parameter is the C associated with the slack variables in the primal objective function, i.e.

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i$$

If you use a kernel fancier than the linear kernel then you will likely have other parameters as well. For instance in the polynomial kernel $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z} + c)^d$ you have to select the shift c and the polynomial degree d . Similarly the rbf kernel

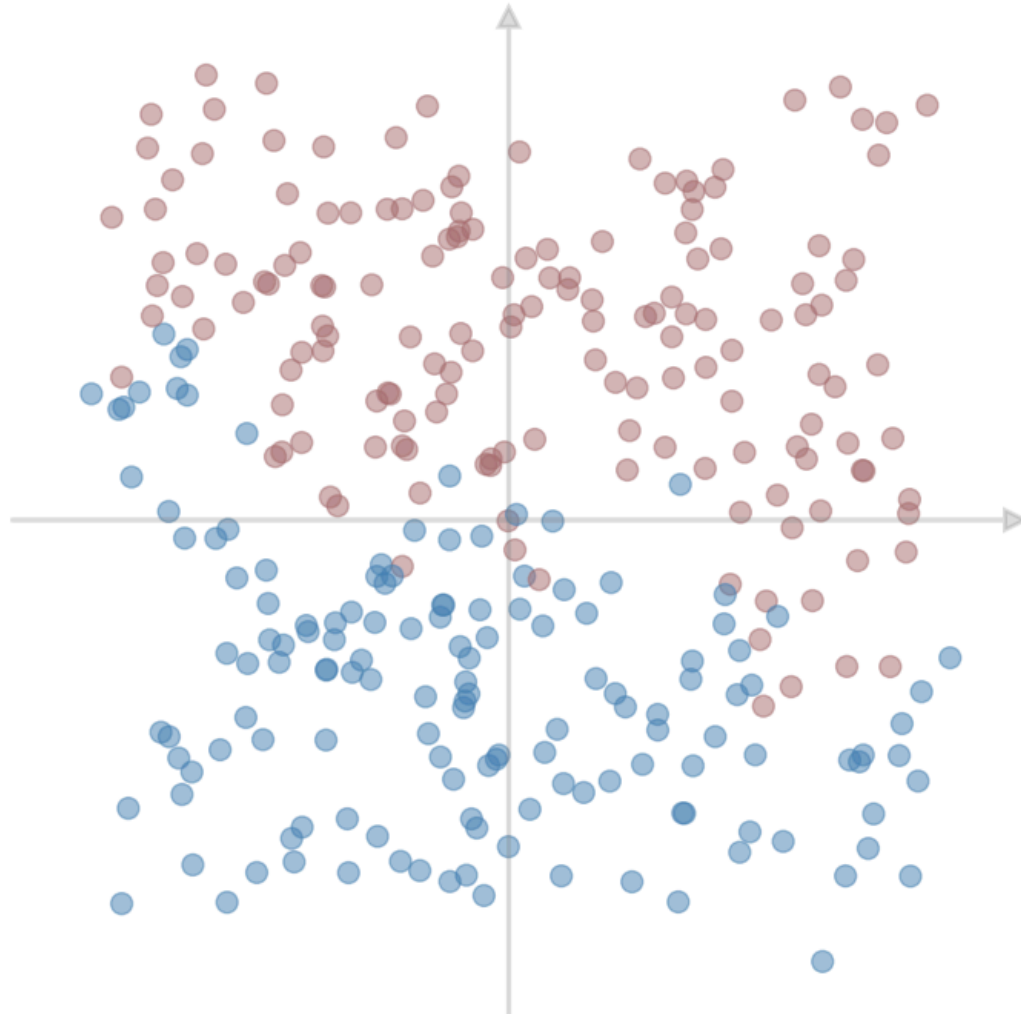
$$K(\mathbf{x}, \mathbf{z}) = \exp[-\gamma \|\mathbf{x} - \mathbf{z}\|^2]$$

has one tuning parameter, namely γ , which controls how fast the similarity measure drops off with distance between $\mathbf{x}$ and $\mathbf{z}$.

For our examples we'll consider the rbf kernel, which gives us two parameters to tune, namely C and γ .

Consider the following two dimensional data

```
In [24]: X, y = part3data(N=300, seed=1235)
         nonlinear_plot(X, y)
```

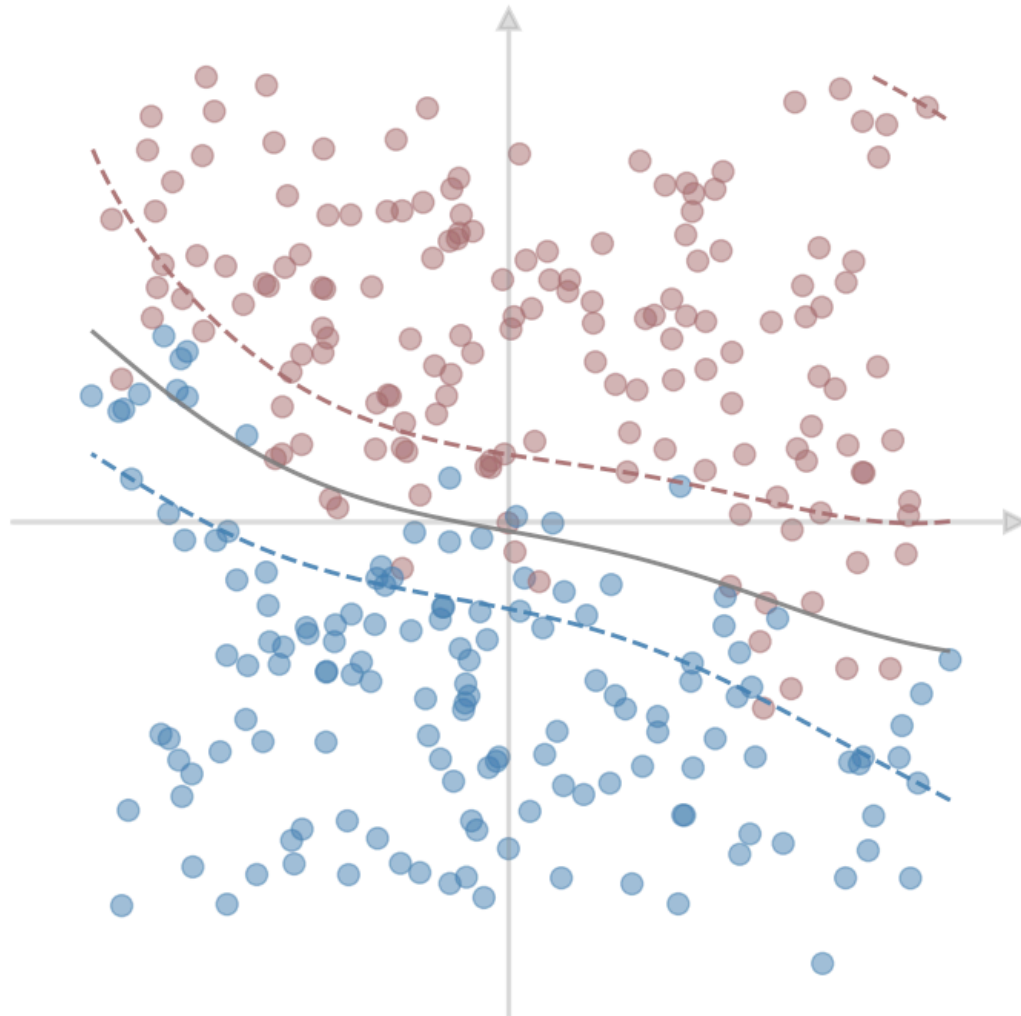


Part A [5 pts]: We can use the method `SVC` from sklearn's `svm` module to fit an SVM with a nonlinear kernel to the data. Go now and look at the documentation. Note that we pass the `kernel="rbf"` parameter to use the RBF kernel. The other two parameters we'll be concerned with are `C` and the RBF parameter `gamma`.

Write some code to fit an SVM with RBF kernel to the data and plot the results. Use the parameter values `C=1` and `gamma=1`.

```
In [28]: from sklearn.svm import SVC

nlsvm= SVC(kernel='rbf', C=1, gamma=1)
nlsvm.fit(X, y)
# YOUR CODE HERE
#raise NotImplementedError()
nonlinear_plot(X, y, nlsvm)
```



Part B [15 pts]: In this part we'll use cross-validation to estimate the validation accuracy achieved by our model. Experiment with the values of the hyperparameters to see if you can get a good validation accuracy. How do the choice of `C` and `gamma` affect the resulting decision boundary?

```
In [32]: from sklearn.model_selection import cross_val_score

# YOUR CODE HERE
#raise NotImplementedError()
# uncomment and update nlsvm and scores
nlsvm = SVC(kernel='rbf', C=10, gamma=0.1)
scores= cross_val_score(nlsvm, X, y, cv=5)
print("cross-val mean-accuracy: {:.3f}".format(np.mean(scores)))

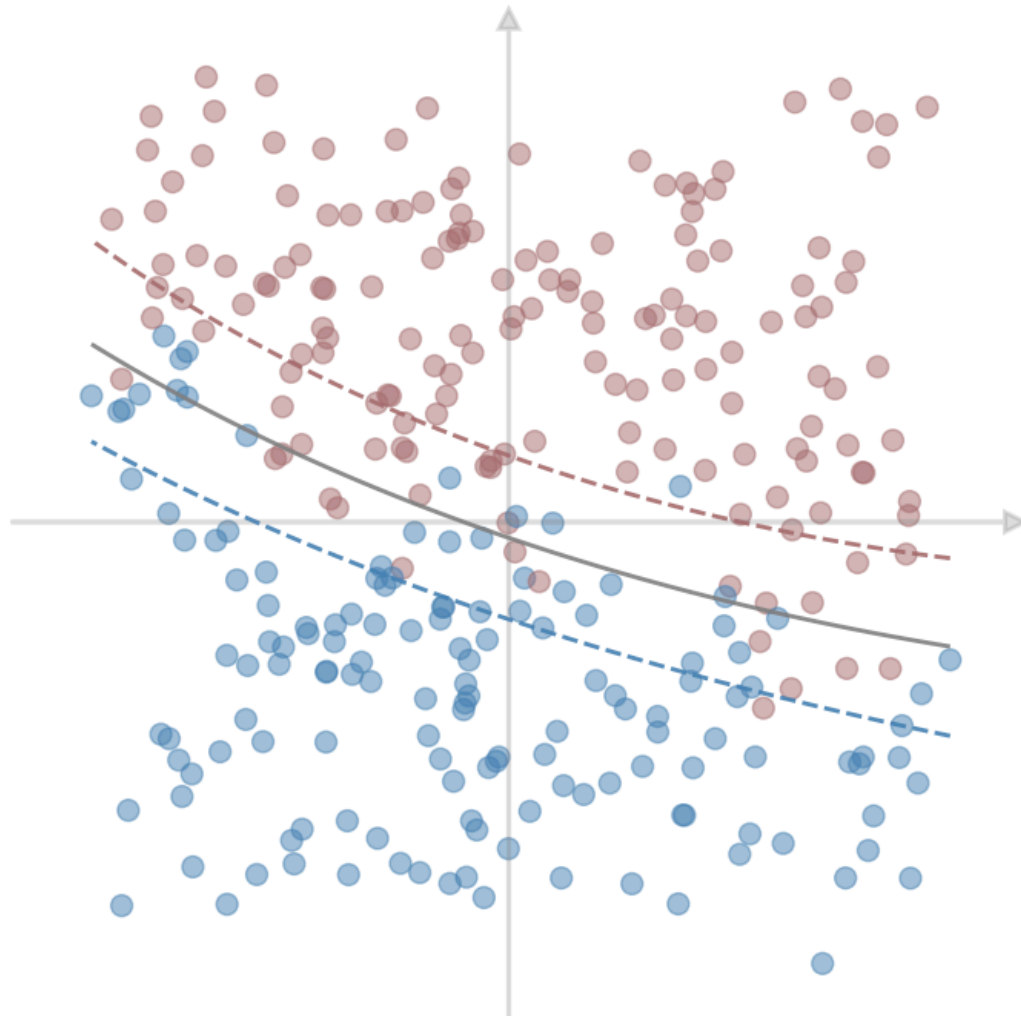
nlsvm.fit(X, y)
nonlinear_plot(X, y, nlsvm)

# C tuning
```

```
# YOUR CODE HERE
#raise NotImplementedError()
for C_val in [0.1, 1, 10, 100]:
    model = SVC(kernel='rbf', C=C_val, gamma=0.1)
    scores = cross_val_score(model, X, y, cv=5)
    print(f'C={C_val}, mean accuracy: {np.mean(scores):.3f}')

# gamma tuning
# YOUR CODE HERE
#raise NotImplementedError()
for gamma_val in [0.01, 0.1, 1, 10]:
    model = SVC(kernel='rbf', C=10, gamma=gamma_val)
    scores = cross_val_score(model, X, y, cv=5)
    print(f'gamma={gamma_val}, mean accuracy: {np.mean(scores):.3f}')
```

```
cross-val mean-accuracy: 0.913
C=0.1, mean accuracy: 0.917
C=1, mean accuracy: 0.907
C=10, mean accuracy: 0.913
C=100, mean accuracy: 0.920
gamma=0.01, mean accuracy: 0.903
gamma=0.1, mean accuracy: 0.913
gamma=1, mean accuracy: 0.933
gamma=10, mean accuracy: 0.943
```



Part C [5 pts]: How does the choice of **kernel** function affect the bias/variance of the model?

The choice of kernel function controls the model's flexibility, which directly affects the bias-variance tradeoff.

Linear kernel has high bias, low variance, and assumes linear separation (underfits).

RBF Kernel has low bias and high variance. A small gamma will have a smoother boundary and higher bias. A larger gamma overfits noise and has high variance.

```
In [ ]: # instructor testing cell
        # YOUR CODE HERE
        raise NotImplementedError()
```

Part 4: Automating the Parameter Search [20 pts]

On the previous problem we were able to choose some OK parameters just by hand-tuning. But in real life (where time is money) it would be better to do something a little more automated. One common thing to do is a **grid-search** over a predefined range of the parameters. In this case you will loop over all possible combinations of parameters, estimate the accuracy of your model using K-Folds cross-validation, and then choose the parameter combination that produces the highest validation accuracy.

Part A [5 pts]: Below is an experiment where we search over a logarithmic range between 2^{-5} and 2^5 for C and a range between 2^{-5} and 2^5 for γ . For the accuracy measure we use K-Folds CV with $K = 3$.

```
In [35]: from sklearn.model_selection import cross_val_score, GridSearchCV
grid=None # ToDo: replace it to proper GridSearchCV object and run the grid

# YOUR CODE HERE
#raise NotImplementedError()

X, y = part3data()

# logarithmic range from 2^-5 to 2^5
param_range = [2 ** i for i in range(-5,6)]

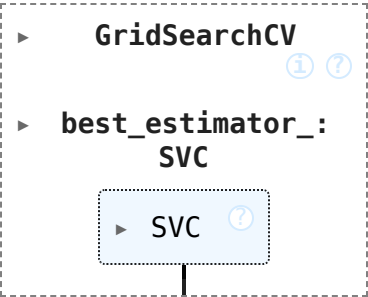
# parameter grid
param_grid = {
    'C': param_range,
    'gamma': param_range
}

svc = SVC(kernel='rbf')

grid = GridSearchCV(svc, param_grid, cv=3)

grid.fit(X, y)
```

```
Out [35]:
```



```

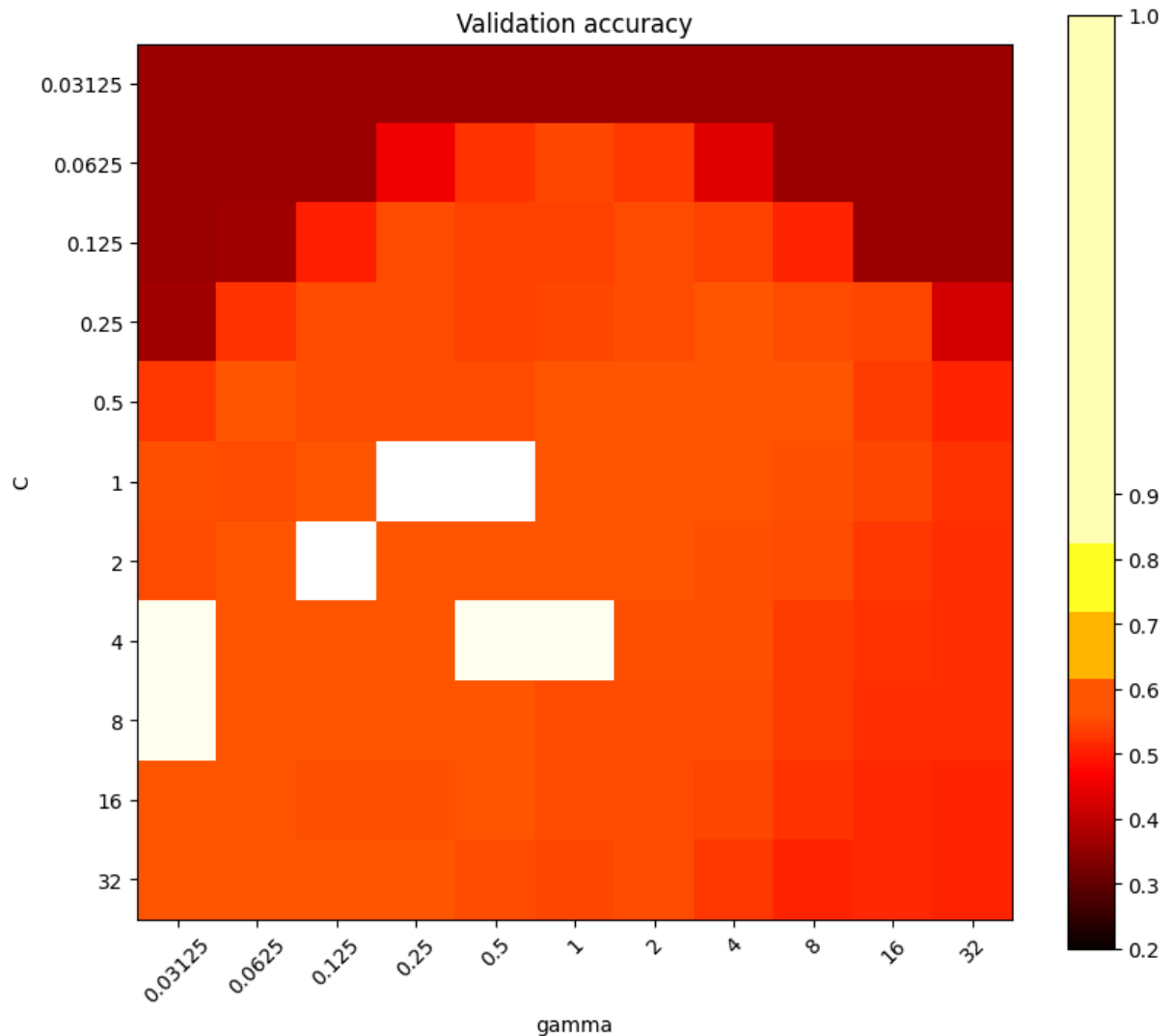
  ▶ GridSearchCV ⓘ ?
    ▶ best_estimator_: SVC
      ▶ SVC ⓘ

```

Part B [5 pts]: The following function will plot a heat-map of the cross-validation accuracies for each combination of parameters. Which combination looks the best?

The best combination of parameters is: {'C': 1, 'gamma': 0.25}. They gave the highest cross-validation accuracy according to the heatmap and GridSearchCV.

```
In [41]: plotSearchGrid(grid)
```

```
In [ ]: # instuctor testing cell
# YOUR CODE HERE
raise NotImplementedError()
```

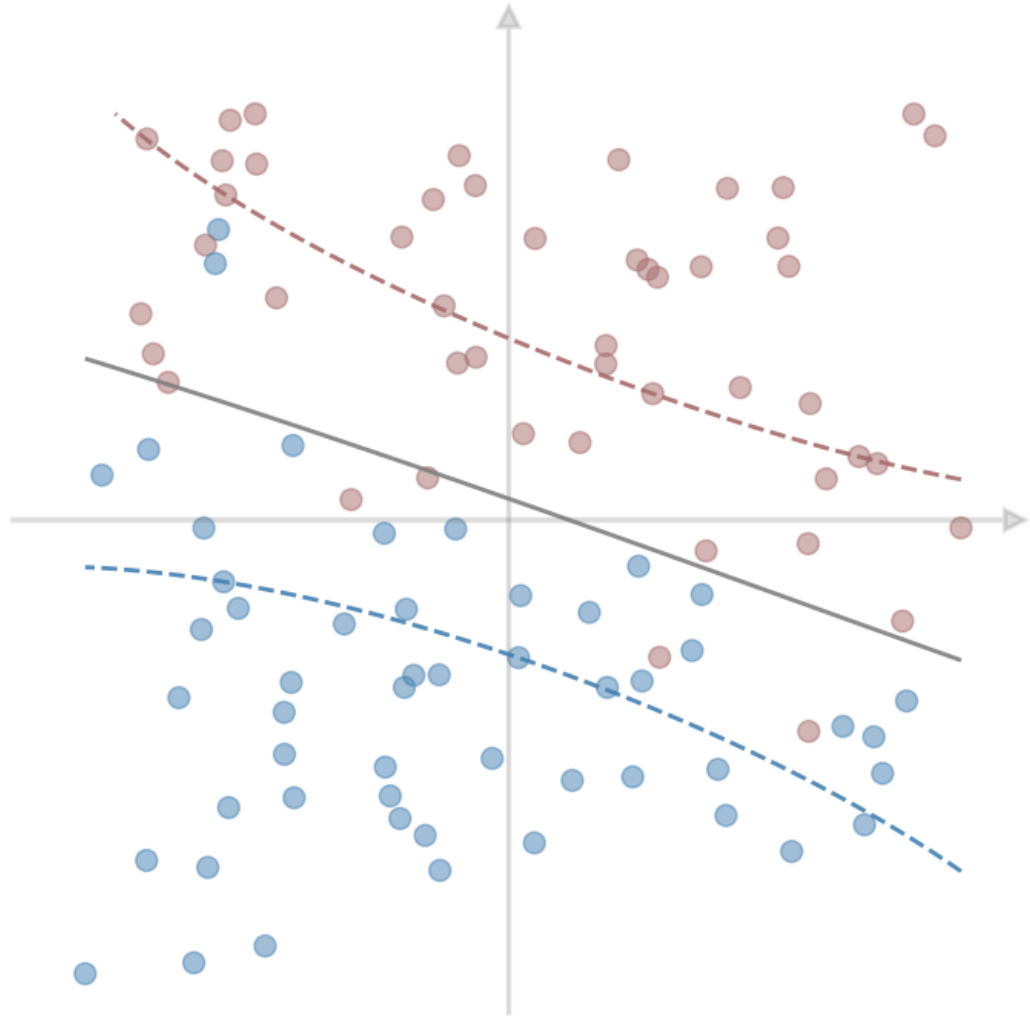
Part C [5 pts]: The GridSearchCV object scores, among other things, the best combination of parameters as well as the cross-validation accuracy achieved with those parameters. Print those quantities for our model.

```
In [37]: # print best paramters and best accuracy
# YOUR CODE HERE
#raise NotImplementedError()
print("Best parameters:", grid.best_params_)
print("Best cross-validation accuracy:", grid.best_score_)
```

Best parameters: {'C': 1, 'gamma': 0.25}
 Best cross-validation accuracy: 0.9298871063576946

Part D [5 pts]: The GridSearchCV object also stores the classifier trained with the best hyperparameters. Pass this best estimator into the `nonlinear_plot` function to view the best decision boundary. What can you tell about the best decision boundary?

```
In [39]: # pass GridSearchCV object best estimator into nonlinear_plot  
# YOUR CODE HERE  
#raise NotImplementedError()  
nonlinear_plot(X, y, clf=grid.best_estimator_)
```



In []: